

Ex. No: 7 Banker's Algorithm

C PROGRAM:

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;
    int allocation[10][10], max[10][10];
    int need[10][10];
    int available[10];
    int finish[10] = {0};
    int safeSeq[10];
    int count = 0;

    printf("Enter Number of Processes: ");
    scanf("%d", &n);

    printf("Enter Number of Resources: ");
    scanf("%d", &m);

    printf("\nEnter Allocation Matrix:
\n");

    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &allocation[i][j]);

    printf("
\n");
```

\nEnter Maximum Matrix:

\n");

```
for(i = 0; i < n; i++)
```

```
for(j = 0; j < m; j++)
```

```
scanf("%d", &max[i][j]);
```

```
printf("
```

\nEnter Available Resources:

\n");

```
for(i = 0; i < m; i++)
```

```
scanf("%d", &available[i]);
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
for(j = 0; j < m; j++)
```

```
{
```

```
need[i][j] = max[i][j]
```

```
- allocation[i][j];
```

```
}
```

```
}
```

```
while(count < n)
```

```
{
```

```
int found = 0;
```

```
for(i = 0; i < n; i++)
```

```

{
if(finish[i] == 0)
{
for(j = 0; j < m; j++)
{
if(need[i][j] > available[j])
break;
}
if(j == m)
{
for(k = 0; k < m; k++)
available[k] += allocation[i][k];
safeSeq[count++] = i;
finish[i] = 1;
found = 1;
}
}
}
if(found == 0)
{
printf("\nSystem is NOT in Safe State\n");
return 0;
}
}
printf("\nSystem is in Safe State\n");
printf("Safe Sequence: ");

```

```

for(i = 0; i < n; i++)
printf("P%d ", safeSeq[i]);
printf("\n");
return 0;
}

```

Output:

```

(kali㉿kali)-[~]
$ nano banker.c

(kali㉿kali)-[~]
$ gcc banker.c -o banker

(kali㉿kali)-[~]
$ ./banker
Enter Number of Processes: 1
Enter Number of Resources: 2

Enter Allocation Matrix:
3
4

Enter Maximum Matrix:
5
6

Enter Available Resources:
8
9

System is in Safe State
Safe Sequence: P0

```

SHELL SCRIPT

```

#!/bin/bash

echo "Enter Number of Processes:"

read n

echo "Enter Safe Sequence (space separated):"

```

```
read -a seq
echo "Safe Sequence is:"
for ((i=0;i<n;i++))
do
echo -n "P${seq[$i]} "
done
echo
echo "System is in Safe State"
```

Output:

```
(kali㉿kali)-[~]
$ nano lab-7.sh

(kali㉿kali)-[~]
$ chmod +x lab-7.sh

(kali㉿kali)-[~]
$ bash lab-7.sh
Enter Number of Processes:
2
Enter Safe Sequence (space separated):
3
Safe Sequence is:
P3 P
System is in Safe State
```